

INFORME DE ARQUITECTURA DE SISTEMA



Asignatura: Programación en Ambiente Web (código 11086).

Docentes: Javier Echaiz (profesor responsable), Santiago Ricci (JTP), Silvio Solari (ayudante de primera), Jorge González (ayudante de primera).

Alumnos:

Axel Hoffmann (legajo 154840).

Facundo Ezequiel Laffont (legajo 105062).

Gonzalo Echeverría Crenna (legajo 195155).

Renzo Martin Robles (legajo 162628).

Arquitectura de la Aplicación PAWPrints

Materia: Programación en Ambiente Web.
Proyecto: Librería PAWPrints
Fecha: Mayo 2026

1. Visión general

PAWPrints es una aplicación web de librería construida en **PHP 8** sin frameworks externos de routing o MVC. El diseño sigue el patrón **MVC simplificado** con **inyección de dependencias**, lo que permite mantener separación de responsabilidades sin acoplar los módulos entre sí.

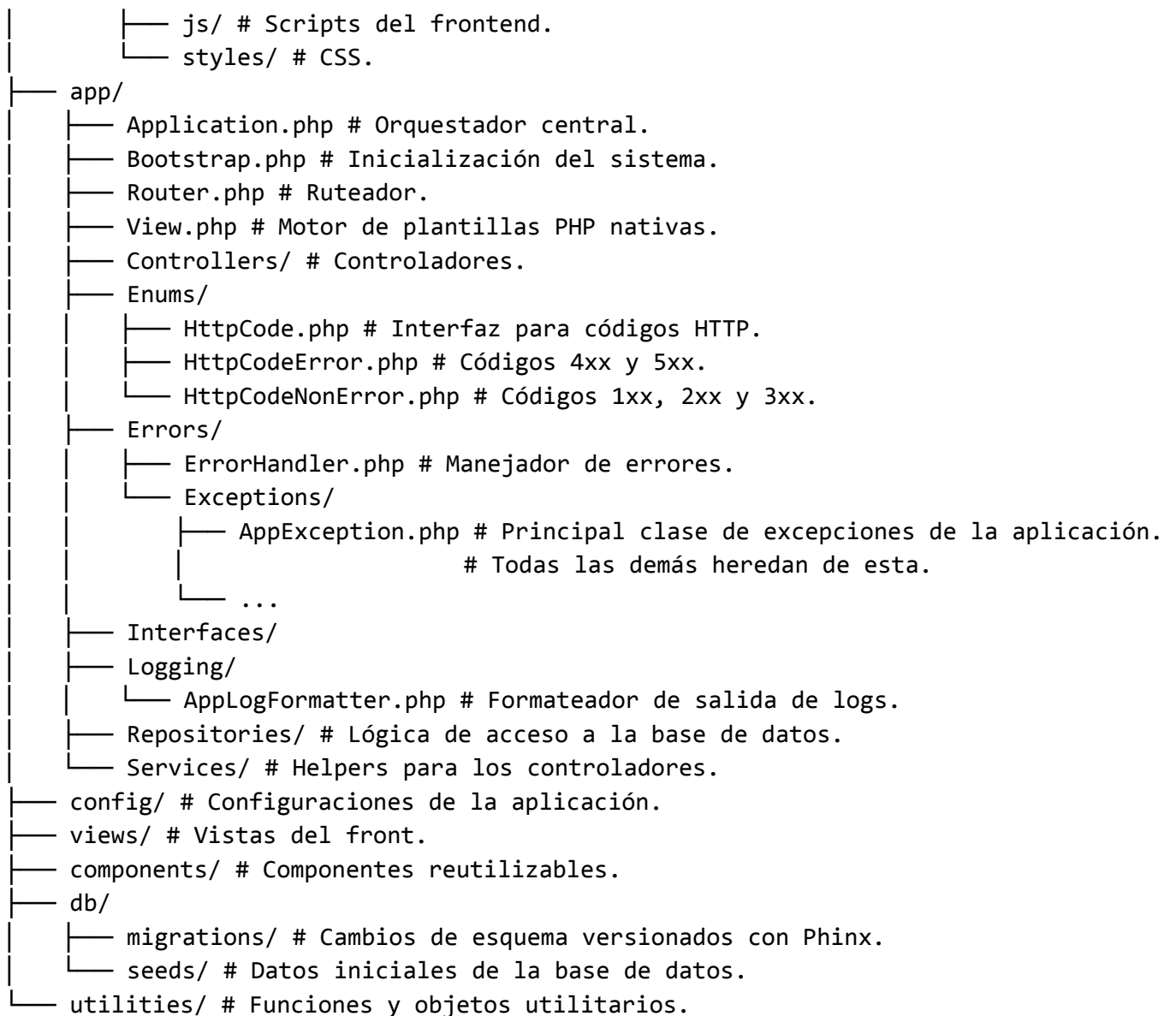
Stack tecnológico

Capa	Tecnología
Lenguaje	PHP 8+
Base de datos	MySQL 8.0
Servidor HTTP	PHP built-in server (dev)
Orquestación	Docker + Docker Compose
Migraciones	Phinx 0.16
DI Container	php-di/php-di 7.1
Logging	monolog/monolog 3.10
Email	phpmailer/phpmailer 6.9
Env vars	vlucas/phpdotenv 5.6
Debug visual	filp/whoops 2.18

2. Estructura general de directorios

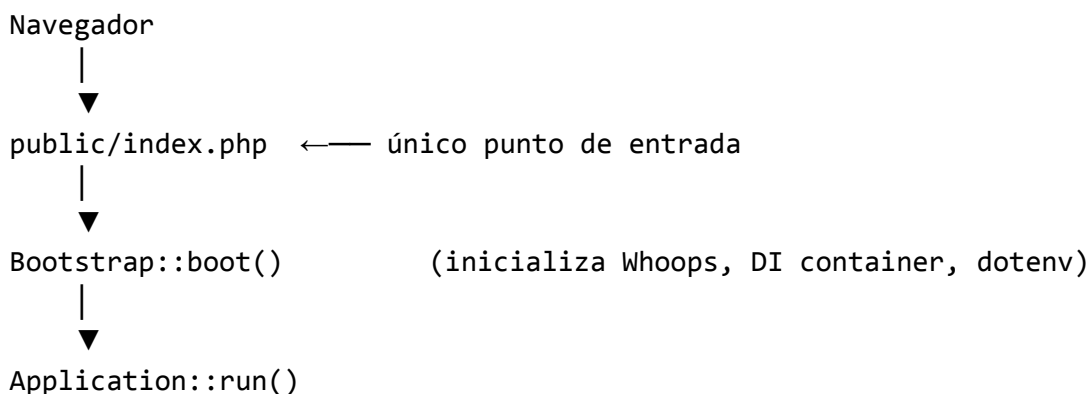
La aplicación está utilizando la siguiente estructura general

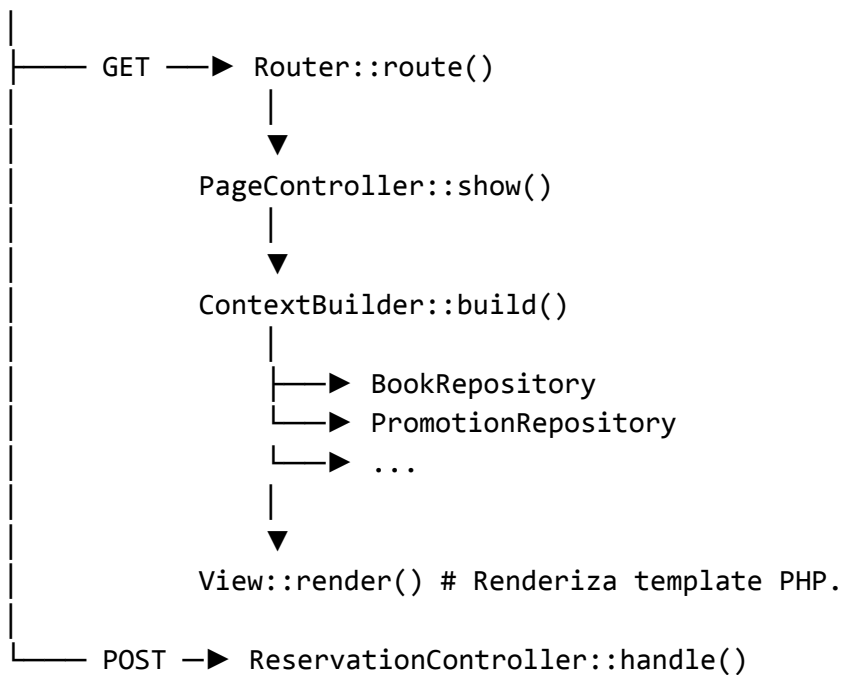
```
PAW-2026-Libreria/  
├── docker-compose.yml # Orquestación de contenedores  
├── Dockerfile # Imagen PHP custom  
└── src/  
    ├── public/  
    │   ├── index.php # Punto de entrada único (front controller).  
    │   └── resources/  
    │       └── images/ # Imágenes de libros.
```



3. Patrón Arquitectónico: MVC con Front Controller

La aplicación implementa el patrón **Front Controller**: todo tráfico HTTP entra por `public/index.php` y desde ahí se despacha al componente correcto. No existe un archivo PHP por página.





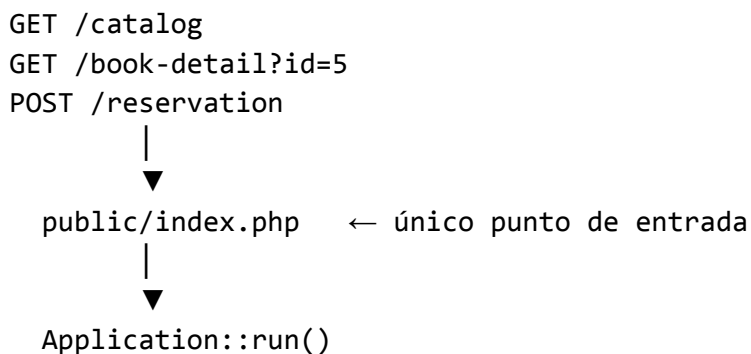
4. Patrones de Diseño

La aplicación aplica varios patrones de diseño clásicos (GoF) y patrones web específicos. Acá se detalla dónde aparece cada uno en el código.

4.1 Front Controller

Dónde: `public/index.php` → `Application::run()`

Todo el tráfico HTTP pasa por un único punto de entrada. `index.php` inicializa el sistema y delega en `Application`, que decide cómo procesar cada request. No hay un `catalog.php`, `home.php`, etc. accesibles directamente.



Beneficio: centraliza autenticación, logging, manejo de errores y bootstrap en un solo lugar.

4.2 MVC (Model-View-Controller) simplificado

Dónde: toda la aplicación.

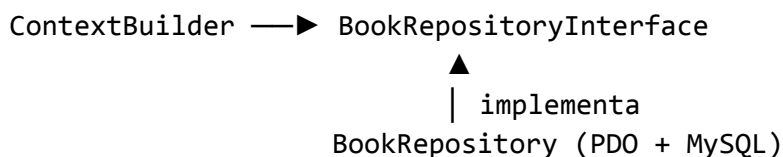
Rol	Clase/s
Model	<code>BookRepository</code> , <code>PromotionRepository</code> (acceso a datos)
View	<code>View.php</code> + templates en <code>views/</code>
Controller	<code>PageController</code> , <code>ReservationController</code>

La versión es "simplificada" porque no hay una clase Model con lógica de negocio pesada: los repositorios retornan arrays PHP y los controllers delegan en `ContextBuilder` para preparar los datos antes de la vista.

4.3 Repository

Dónde: `Interfaces/BookRepositoryInterface`, `Repositories/BookRepository`, `Interfaces/PromotionRepositoryInterface`, `Repositories/PromotionRepository`

Separa la lógica de acceso a datos del resto de la aplicación mediante una interfaz. Los controllers y `ContextBuilder` conocen la **interfaz**, no la implementación concreta con PDO.



Consecuencia práctica: para testear `ContextBuilder` se puede inyectar un `BookRepositoryInterface` falso sin tocar la base de datos. Para migrar a PostgreSQL, solo se cambia la implementación concreta y se re-registra en `container.php`.

4.4 Dependency Injection + Service Locator (IoC Container)

Dónde: `config/container.php` + `Bootstrap::buildContainer()` + `php-di/php-di`

En lugar de que cada clase cree sus dependencias con `new`, el **contenedor DI** las instancia y las inyecta. `container.php` actúa como la única fuente de verdad sobre cómo se construye cada componente:

```
// container.php – las clases no saben cómo se construyen entre sí
BookRepositoryInterface::class => DI\autowire(BookRepository::class),
LoggerInterface::class        => DI\factory(function() { ... }),
```

```
PDO::class => DI\factory(function() { ... } ),
```

El contenedor resuelve el árbol de dependencias automáticamente (autowiring): si `BookRepository` necesita `PDO` y `LoggerInterface`, PHP-DI los inyecta sin código adicional.

4.5 Builder

Dónde: `Services/ContextBuilder.php`

`ContextBuilder` construye el contexto de datos (el array de variables) que recibe cada vista. El método público `build($page)` despacha a un método privado especializado según la página:

```
ContextBuilder::build('catalog')
└─▶ buildCatalogContext()
    ├──▶ buildCatalogFilters() ← sub-builder de filtros
    ├──▶ BookRepository::findAllGenres()
    ├──▶ BookRepository::findAllAuthors()
    ├──▶ BookRepository::countAll(filters)
    └─▶ BookRepository::findAll(offset, limit, filters)

ContextBuilder::build('home-page')
└─▶ buildHomePageContext()
    ├──▶ BookRepository::findAllGroupedByGenre()
    └─▶ PromotionRepository::findAll()
```

Cada `build*` construye un producto diferente (el contexto de su vista) con pasos propios. El controller solo llama a `build()` y recibe el contexto listo para renderizar.

4.6 Template View

Dónde: `View.php` + archivos en `views/`

`View::render($page, $context)` usa `extract($context)` para llevar las variables del contexto al scope del template, y luego hace `require` del archivo PHP correspondiente:

```
// View::render()
extract($context);           // $books, $filters, $genres, etc. quedan disponibles
require "views/{$page}.php"; // el template los usa directamente
```

Cada archivo en `views/` es un template que usa PHP nativo (sin motor externo como Twig o Blade). Los componentes reutilizables (`header.php`, `footer.php`, `head.php`) se incluyen con `require` desde las vistas.

4.7 Value Object (PHP Enums)

Dónde: `Enums/HttpCode.php`, `Enums/HttpCodeError.php`, `Enums/HttpCodeNonError.php`

Los códigos HTTP se modelan como enums que encapsulan tanto el código numérico como el mensaje en español, garantizando que solo existan valores válidos:

```
enum HttpCodeError implements HttpCode {
    case NotFound;
    case InternalServerError;
    // ...
    public function toInt(): int { ... }
    public function getMessage(): string { ... }
}
```

`HttpException` recibe un `HttpCodeError`, no un entero suelto. Esto hace imposible lanzar una excepción con un código HTTP inventado.

4.8 PRG — Post/Redirect/Get

Dónde: `Controllers/ReservationController.php`

Cuando el formulario de reserva se envía y procesa exitosamente, el controller no renderiza la vista directamente: emite un redirect HTTP 302 a `GET /reservation?enviada=1`.

```
POST /reservation → procesamiento → header('Location: /reservation?enviada=1')
                                     |
                                     GET /reservation?enviada=1
                                     |
                                     View::render (muestra mensaje de éxito)
```

Por qué: si el controller respondiera directamente con HTML al POST, al refrescar el navegador reenviaría el formulario. Con PRG, el refresh repite un GET inofensivo.

5. Problemáticas y Módulos con PSRs

5.1 Análisis de peticiones HTTP

Módulo: `Application` (`Application.php`)

`Application::run()` lee directamente las superglobales `$_SERVER['REQUEST_METHOD']` y `$_SERVER['REQUEST_URI']` para extraer el método y la ruta entrante. El URI se limpia de parámetros query (`parse_url`) antes de compararse con la tabla de rutas.

Además maneja un caso especial: `GET /catalog?export=csv` genera y descarga el catálogo completo en formato CSV sin pasar por la vista HTML.

PSR relacionado: PSR-7 — HTTP Message Interface

Define interfaces estándar para `RequestInterface` y `ServerRequestInterface`. Adoptarlo permitiría encapsular la petición en un objeto inmutable en lugar de acceder a superglobales directamente. Hoy esto funciona bien para la escala del proyecto, pero a medida que crezca la cantidad de rutas y middlewares, PSR-7 facilitaría la extensión.

5.2 Mapeo de URLs a funcionalidades

Módulos: `Router` (`Router.php`) + `Application` (`Application.php`)

`Router` mantiene una tabla de rutas registradas con `addRoute($path, $page, $title)` y las resuelve con `route($uri)`. La resolución es matching exacto (sin regex ni parámetros en la URL: los IDs se pasan por query string).

Rutas disponibles:

Path	Página	Descripción
<code>/</code>	home-page	Página de inicio con carruseles
<code>/catalog</code>	catalog	Catálogo con filtros y paginación
<code>/book-detail</code>	book-detail	Detalle de un libro (requiere <code>?id=</code>)
<code>/reservation</code>	reservation	Formulario de reserva
<code>/promotions</code>	promotions	Promociones activas
<code>/about-us</code>	about-us	Información del equipo

Si la URI no coincide con ninguna ruta, `Application` lanza `HttpException` con código 404.

5.3 Generación de respuestas HTTP

Módulos:

- **View** (**View.php**): renderiza el HTML como cuerpo de la respuesta.
- **ErrorHandler** (**ErrorHandler.php**): establece el código HTTP de error y renderiza la vista de error.
- **ReservationController** (**ReservationController.php**): decide si re-renderiza el formulario o redirige.

View::render() hace **require** del template PHP pasándole las variables de contexto mediante **extract()**. No usa un motor de templates externo como Twig.

ErrorHandler diferencia dos tipos de error:

- **handleHttpError(HttpErrorException)**: para errores de dominio (4XX y 5XX)
- **handleAppError(Throwable)**: para errores inesperados, responde siempre con 500

Los códigos HTTP están modelados como enums PHP:

HttpCode (interface)

- └─ **HttpCodeError**: 4XX y 5XX.
- └─ **HttpCodeNonError**: 1XX, 2XX y 3XX.

PSR relacionado: PSR-7 — HTTP Message Interface (ResponseInterface)

Actualmente se usan funciones nativas (**http_response_code**, **header**, **echo**). Encapsular la respuesta en un objeto **ResponseInterface** haría el código más testeable y desacoplado del output buffer.

5.4 Logging

Módulos:

- **AppLogFormatter** (**Logging/AppLogFormatter.php**): formateador personalizado.
- Configuración en **container.php**: instancia el logger con handlers y processors.

El logger se configura con tres capas:

1. **StreamHandler**: escribe a **app.log** con nivel **DEBUG**
2. **IntrospectionProcessor**: agrega automáticamente archivo, línea, clase y método al contexto.
3. **Processor custom**: le da formato personalizado a la salida del log.

AppLogFormatter extiende **NormalizerFormatter** de Monolog y produce una salida estructurada con bloques de contexto.

Todo el código del dominio recibe **Psr\Log\LoggerInterface** inyectado (no **Monolog\Logger** directamente), lo que permite reemplazar la implementación sin modificar nada fuera del container.

PSR relacionado: PSR-3 — Logger Interface adoptado

Ya implementado. `LoggerInterface` está inyectado en `BookRepository`, `ReservationController`, `EmailService` y `PageController`.

5.5 Persistencia

Módulos:

- `Interfaces/BookRepositoryInterface` y `Interfaces/PromotionRepositoryInterface`: contratos del dominio.
- `Repositories/BookRepository` y `Repositories/PromotionRepository`: implementaciones con PDO.
- Configuración de PDO en `container.php`: modo exception, fetch ASSOC.
- Migraciones en `db/migrations/` y seeds en `db/seeds/`.

PSR relacionado: No existe un PSR específico para repositorios, pero el patrón Repository permite intercambiar la implementación (por ejemplo, cambiar MySQL por PostgreSQL) sin tocar la lógica de negocio.

5.6 Configuración e Inyección de Dependencias

Módulos:

- `Bootstrap` (`Bootstrap.php`): punto de entrada para la inicialización.
- `container.php`: define todas las dependencias.
- Variables de entorno: `.env` (cargado con `phpdotenv`).

`Bootstrap::buildContainer()` construye un `ContainerBuilder` de PHP-DI, compila el container con las definiciones de `container.php` y lo retorna como `Psr\Container\ContainerInterface`.

PSR relacionado: PSR-11 — Container Interface (adoptado)

`Bootstrap::buildContainer()` retorna `Psr\Container\ContainerInterface`. La implementación concreta es `php-di/php-di`, reemplazable sin cambiar el código que consume el container.

5.7 Generación de representaciones de la información

Módulos:

- `View` (`View.php`): genera HTML con templates PHP nativos.

- `EmailService` (`EmailService.php`): genera emails en HTML + texto plano.
- `ContextBuilder` (`ContextBuilder.php`): construye el contexto de datos para cada vista.

`ContextBuilder::build()` actúa como **director**: según la página solicitada, llama al método de construcción correcto.

`EmailService` envía emails de confirmación de reserva a través de SMTP (PHPMailer). Genera tanto versión HTML como texto plano alternativo y loguea el resultado.

PSR relacionado: No existe un PSR específico para templating. PSR-7 aplica conceptualmente al encapsular el cuerpo de la respuesta independientemente del Content-Type (`text/html`, futura `application/pdf`, etc.).

6. Esquema de la Base de Datos

Tabla `books`

```
CREATE TABLE books (
  id          INT PRIMARY KEY AUTO_INCREMENT,
  title       VARCHAR(255),
  author      VARCHAR(255),
  genre       VARCHAR(100),
  description  TEXT,
  image       VARCHAR(500),  -- formato: "maxWidth:img-chica.png;img-grande.png"
  isbn        VARCHAR(13) UNIQUE,
  price       DECIMAL(10,2),
  stock       INT DEFAULT 0,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
)
```

El campo `image` tiene un formato personalizado para soportar **imágenes responsive**:

```
599:placeholder-libro-chica.png;placeholder-libro-grande.png
   ↑           ↑           ↑
maxWidth      src móvil    src escritorio
```

Se usa en el elemento HTML `<picture>` con `<source media="(max-width:Npx)">`.

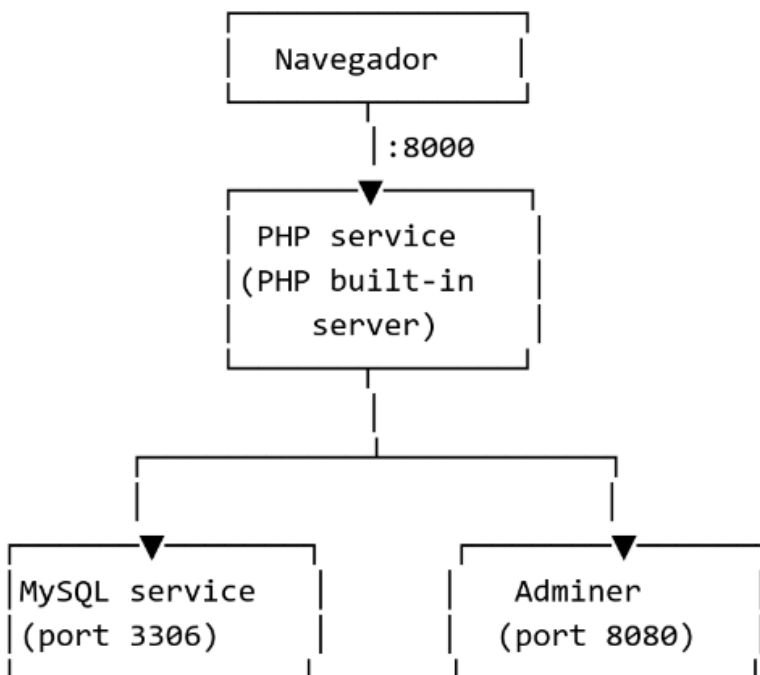
Tabla `promotions`

```
CREATE TABLE promotions (  
  id          INT PRIMARY KEY AUTO_INCREMENT,  
  description VARCHAR(255),  
  image       VARCHAR(255),  
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
)
```

7. Configuración y Deployment

El sistema hace uso de un archivo de configuración `.env`, que no se comparte en el repositorio (ya que es información sensible) y el cual contiene las variables que formarán parte del entorno de la aplicación.

Docker Compose



El servicio PHP al iniciar:

1. Instala dependencias con `composer install`
2. Ejecuta migraciones: `composer db:migrate`
3. Ejecuta seeds: `composer db:seed`
4. Levanta el servidor: `php -S 0.0.0.0:8000 -t public/`